Aaron Balke

aabalke/gojit: Revived Jit Compiler in Go

aabalke/gojit is a revival of nelhage/gojit, providing JIT compilation for Go version 1.17+. The major functionality this fork adds is the ability for Go functions to be called from jit code, without causing errors from stack checks during garbage collection or stack growth.

March 18, 2026

go jit

Important Links

aabalke/gojit

aabalke/guac

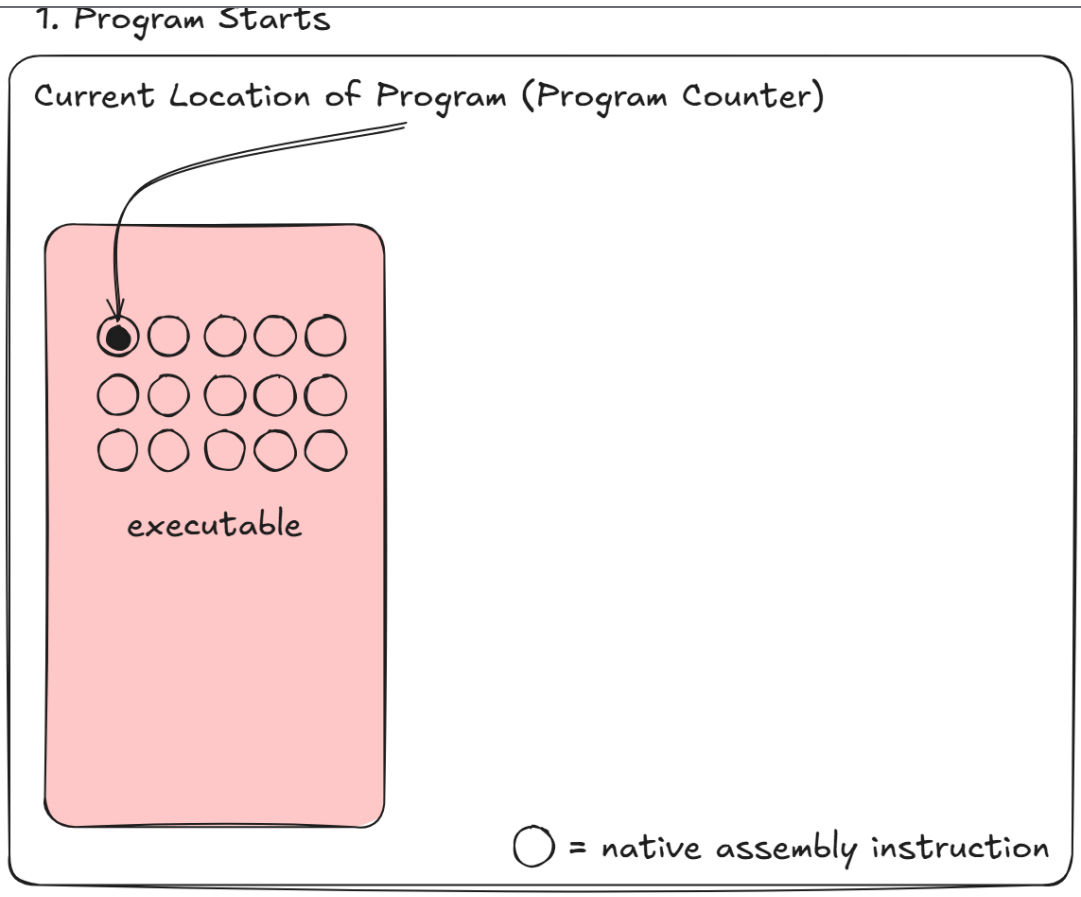
aabalke/jit-proof-of-concept

Just-in-Time Compilation

Most programs use Ahead-of-time (AOT) compilation, the creation of an entire binary or executable before runtime. Often, you compile or build an exe file in your programming language of choice, and all instructions that the program will run are included in that binary. Just-in-Time (JIT) Compilation differs in that the instructions and executable code are created and compiled at runtime, after the program has started. For most situations, this is overly complicated and not necessary; however, for programs that require high performance and have code paths that change significantly based on input files, it is a massive speed improvement. The two main places Jit compilation is used are language interpreters and emulators.

In these programs, constant analysis is made, looking for common code paths, either through emulated instruction blocks or interpreted token blocks. If certain thresholds are met, often a block of instructions or tokens is run hundreds of times, then this block is compiled into machine code. In the future, when the program reaches this block, the compiled machine code is run in place of the original version. This is faster because the new compiled version only decodes the instruction or token block at compilation and handles a majority of conditional statements at compilation. A block of instructions or tokens that is run 10,000 times using the original, not jitted version, would require the block to be decoded and all conditional statements run 10,000 times. While a jitted version would only require the decoding and conditional statements to be run once at compilation. When this is extrapolated to emulating billions of instructions or tokens per second, there is usually a 2-5x speed improvement.

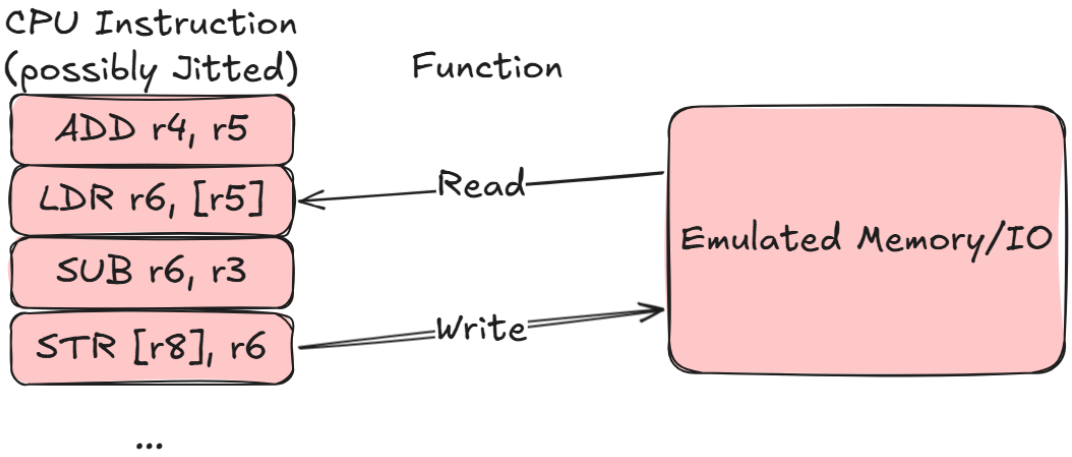
Aaron Balke



Go

Go differs from lower-level languages such as C, C++, and Zig in that it includes a garbage collector, limits direct memory management, and has a strict Application Binary Interface (ABI). There is support for Plan9 Assembly and C interop through Cgo; however, these are limited. Go assembly specifically requires AOT compilation to maintain the Go internal ABI specification. This specification is rigid because of the garbage collector. For the garbage collector to function, the stack must maintain a specific shape and alignment, for argument spill space, stack-assigned results, stack-assigned arguments, and a receiver. This is important during garbage collection and when stack growth occurs. If the stack shape is not maintained, then the program will crash.

Aaron Balke



An additional reason Go is difficult for JIT compilation is that it has 2 ABIs. ABI0 is the original ABI, and ABIInternal (ABI1) is the updated one. The major difference between the 2 is that ABI0 uses the stack for all function arguments and results. In contrast, ABIInternal uses registers for most arguments and results, using the stack only when arguments and/or results exceed the available registers. This difference makes ABIInternal 5% faster, since it is not required to reach out to memory for starting or finishing functions. As the name suggests, ABIInternal is used internally within the language, appearing in the Go executable since Go 1.17; however, ABI0 is still the public-facing ABI, appearing in the "hand-rolled" Go assembly that a Go user would interact with. If an ABI0 assembly function needs to interact with an ABIInternal assembly function, Go will insert a transparent ABI wrapper, converting one ABI into the other.

Purpose

The overlap between JIT compilation and the Go ABIs is why building a JIT in Go is difficult, and why proper handling of Go functions called by JIT code has not been implemented in modern versions. This is important because, when emulating instructions or parsing tokens, the jitted code sometimes must call unjitted functions. For example, in an emulator, with simulated memory, a load from memory address (LDR, or MOV) instruction would need to call a "read memory" function inside the emulator, which is usually a separate function or method.

The following is a proof of concept to demonstrate the solution to this problem. It is broken into 3 sections: a basic JIT compiler, a JIT compiler demonstrating improper ABI handling, and a JIT compiler with proper ABI handling. This section is heavily based on "Calling Go funcs from asm and JITed code" by Iskander Sharipov; however, Sharipov's version was built for Go <1.17 and does not work with the modern ABI0 and ABIInternal situation. All 3 sections are written in version 1.26; however, they should work from 1.17+, when the ABI0 and ABIInternal specifications were introduced. This demonstration is also only written for x86/amd64 instruction set systems, as gojit only supports x86/amd64 at the time of writing.

Proof of Concept: Part 1

The major steps to build a Jit compiler in Go are:

Aaron Balke

2. Insert the desired JIT instructions into that memory.
3. Call a Go Assembly trampoline that jumps to the JIT instructions. After the JIT instructions are executed, the trampoline returns to the initializing Go program.

This is a basic example, where the JIT code moves 0xDEADBEEF into the location of the “c” variable.

Aaron Balke

```go
package main

import (
    "fmt"
    "unsafe"
)

func main() {

    c := uint64(0)
    addr := uint64(uintptr(unsafe.Pointer(&c)))

    code := []byte{
        // MOVABSQ &c, RCX  — load the address of c into RCX
        0x48, 0xB9,
        byte(addr),
        byte(addr >> 8),
        byte(addr >> 16),
        byte(addr >> 24),
        byte(addr >> 32),
        byte(addr >> 40),
        byte(addr >> 48),
        byte(addr >> 56),

        // MOVL $0xDEADBEEF, [RCX]  — move imm32 into memory at addr in RCX
        0xC7, 0x01,
        0xEF, 0xBE, 0xAD, 0xDE, // 0xDEADBEEF in little-endian

        // RET
        0xC3,
    }

    executable, err := mmapExecutable(len(code))
    if err != nil {
        panic(err)
    }

    defer munmapExecutable(executable)

    copy(executable, code)

    callJIT(&executable[0])
    fmt.Printf("C %08X\n", c)
}

// asm stub
func callJIT(code *byte)


// jit_amd64.s
TEXT ·callJIT(SB), 0, $0-8
    MOVQ code+0(FP), AX
    JMP AX
```

- "c" will be the variable we want mutated by the JIT code.
- "addr" will be the pointer to "c", converted into a uint64. This is required to strip away golang safety requirements. "uintptr" is equivalent to C void pointers.
- "code" will be the actual assembly code we will place in the executable JIT memory. For this demonstration, we have 3 instructions.
    - MOVABSQ &c, RCX: Moves the ptr of "c" into the RCX register
    - MOVL $0xDEADBEEF, [RCX]: Moves 0xDEADBEEF into the memory location addressed by RCX
    - RET: returns from the current Call

Secondly, the executable memory where our JIT code will live is created. And the releasing of that memory is deferred at this time: at any end to this function, the memory will be released. Then the instructions we built in the "code" variable are copied into the executable JIT memory.

The final step is to call the JIT code, which requires a piece of Go Assembly to act as a trampoline between the Go code and the JIT code. The "jit_amd64.s" assembly code is this trampoline; it connects to Go through the "function callJIT(code *byte)" stub, which acts as an alias for the Go assembly code. callJIT does 2 things: it moves the first argument of the callJIT function into AX. This is a pointer to the first byte of the JIT code. Then it jumps to the location in AX, starting our JIT code.

The additional assembly declarations are:

- TEXT: declares a function
- (SB): a pseudo-register for the base address of the program
- 0: no flags
- $0-8: bytes required for the local stack frame and bytes required for arguments and return values. 8 is required for our single uint64 argument.

Below are simple boilerplate functions to receive the heap memory for JIT instructions in both Windows and Linux.

```
// unix.go
//go:build linux

package main

import (
    "syscall"
    "unsafe"
)

func mmapExecutable(length int) ([]byte, error) {
    const (
        addr  = 0
        prot  = syscall.PROT_READ | syscall.PROT_WRITE | syscall.PROT_EXEC
        flags = syscall.MAP_PRIVATE | syscall.MAP_ANON
        fd    = 0
        off   = 0
    )

    ptr, _, err := syscall.Syscall6(
```

Aaron Balke

```
    if err != 0 {
        return nil, err
    }

    // Build a Go slice backed by the allocated memory
    slice := unsafe.Slice((*byte)(unsafe.Pointer(ptr)), length)
    return slice, nil
}

func munmapExecutable(_ []byte) error {
    return nil
}
```

```
// win.go
//go:build windows

package main

import (
    "fmt"
    "syscall"
    "unsafe"
)

var (
    kernel32         = syscall.NewLazyDLL("kernel32.dll")
    procVirtualAlloc = kernel32.NewProc("VirtualAlloc")
    procVirtualFree  = kernel32.NewProc("VirtualFree")
)

const (
    MEM_COMMIT  = 0x1000
    MEM_RESERVE = 0x2000
    MEM_RELEASE = 0x8000

    PAGE_EXECUTE_READWRITE = 0x40
)

func mmapExecutable(length int) ([]byte, error) {
    ptr, _, err := procVirtualAlloc.Call(
        0,
        uintptr(length),
        MEM_COMMIT|MEM_RESERVE,
        PAGE_EXECUTE_READWRITE,
    )
    if ptr == 0 {
        return nil, fmt.Errorf("VirtualAlloc failed: %w", err)
    }

    // Build a Go slice backed by the allocated memory
    slice := unsafe.Slice((*byte)(unsafe.Pointer(ptr)), length)
    return slice, nil
}
```

```
        return nil
    }

    addr := uintptr(unsafe.Pointer(&b[0]))
    r, _, err := procVirtualFree.Call(
        addr,
        0,
        MEM_RELEASE,
    )
    if r == 0 {
        return fmt.Errorf("VirtualFree failed: %w", err)
    }
    return nil
}
```

Proof of Concept: Part 2

In languages that do not have to adhere to a strict ABI, the following implementation would work for calling JIT compilation code that calls native language code. However, as mentioned previously, this does not work in Go when garbage collection occurs or other stack-related functions.

```
// main.go
package main

import (
    "reflect"
    "runtime"
)

func main() {
    a := funcAddr(goFunction)

    code := []byte{
        // MOVABSQ a, RAX
        0x48, 0xB8,
        byte(a),
        byte(a >> 8),
        byte(a >> 16),
        byte(a >> 24),
        byte(a >> 32),
        byte(a >> 40),
        byte(a >> 48),
        byte(a >> 56),
        // CALL AX
        0xff, 0xd0,
        // RET
        0xc3,
    }

    executable, err := mmapExecutable(len(code))
    if err != nil {
```

```
    defer munmapExecutable(executable)

    copy(executable, code)
    callJIT(&executable[0])
}

func goFunction() {
    println("called from jit code")
    runtime.GC() // the line that causes the stack functions which break the jit
}

// asm stubs
func callJIT(code *byte)

func funcAddr(f any) uintptr {
    v := reflect.ValueOf(f)
    if v.Kind() != reflect.Func {
        panic("funcAddr: not a func")
    }
    return v.Pointer()
}
```

This implementation has 3 main parts: the JIT code being updated, goFunction being called, and funcAddr, a helper for finding the location of goFunction in memory.

At initial compilation, goFunction is compiled into the program. At runtime, funcAddr is called. funcAddr uses reflection to get the function pointer to goFunction and sets “a” to the pointer.

The “code” variable has been changed from our previous version. Previously, we used our JIT code to update a variable value; this time, we will use it to call a Go function - goFunction. Since we have the pointer to goFunction, we will use “MOVABSQ a, RAX” to move it into register RAX. Then we will jump into it by calling RAX.

At this time, goFunction runs. If a stack-related function is not called, it will return with no errors. However, if a stack-related function like runtime.GC(), which forces garbage collection, is called; it will crash with an error, usually “fatal error: unexpected signal during runtime execution” or “unexpected return pc”.

Proof of Concept: Part 3

Fixing this problem requires a few changes; most importantly, we will need to call Go functions from locations the Go program will recognize, not arbitrary JIT code. We can tack onto our Go assembly a label for handling Go function calls. This will trick the Go runtime into believing we are calling from a valid Go function. This will occur after the JMP instruction for entering the JIT code, which will always have a RET within it. This means the label for handling Go function calls will never be implicitly called; just explicitly called when we want to trick the runtime.

```
#include "funcdata.h"
```

```
    NO_LOCAL_POINTERS
    MOVQ code+0(FP), AX
    JMP AX
gocall:
    CALL CX
    JMP (SP)
```

- $8-8 is changed, since we need 8 frame bytes to write the origin return address.
- NO_LOCAL_POINTERS is due to the CALL and non-zero frame size (requires funcdata.h import)

In versions of Go <1.17, this would be the point at which things begin to simplify. Using the funcAddr function, a pointer to callJIT can be found, and the offset of the gocall label can be added; however, in modern versions of Go with both ABIs, there is an additional twist. As mentioned previously, Go ABI0 will have a wrapper inserted for it to interop with ABIInternal; this occurs with our callJIT function. If you attempt to get a pointer at runtime to callJIT using our funcAddr, it will actually return the wrapper's pointer, not the original callJIT function pointer. To remedy this, another assembly function is added:

```
// Helper: func callJITImplAddr() uintptr
// Returns the address of the ABI0 implementation symbol.
TEXT ·callJITImplAddr(SB), 0, $0-8
    NO_LOCAL_POINTERS
    MOVQ $·callJIT(SB), AX  // address of ABI0 impl, not trampoline
    MOVQ AX, ret+0(FP)
    RET
```

This function pulls the pointer to the original ABI0 callJIT, not the ABIInternal wrapper. It then returns this on register AX, the first and only return value. From this point, we have a pointer to callJIT, and need to add an offset for the gocall label.

```
// asm stub
func callJITImplAddr() uintptr

func getCallAddr() uintptr {

    impl := callJITImplAddr()

    // most offsets seem to be between 30 - 40
    b := unsafe.Slice((*byte)(unsafe.Pointer(impl)), 0x60)

    // equal to call cx
    label := []byte{0xFF, 0xD1}

    // get index of CALL CX
    offset := bytes.Index(b, label)

    return impl + uintptr(offset)
}
```

Aaron Balke

- “impl” is the address of callJIT as mentioned previously
- “b” is an array of the first few bytes in callJIT
- “label” is the gocall label, in our case CALL CX
- “offset” is the calculated offset from “impl”, calculated by searching for the first CALL CX
- getCallAddr return callJIT + offset of gocall

Aaron Balke

```
    a := funcAddr(goFunction)
    j := getCallAddr()

    code := []byte{
        // MOVABSQ funcAddr(f), CX
        0x48, 0xB9,
        byte(a),
        byte(a >> 8),
        byte(a >> 16),
        byte(a >> 24),
        byte(a >> 32),
        byte(a >> 40),
        byte(a >> 48),
        byte(a >> 56),
        // MOVABSQ funcAddr(callJIT)+offset (gocall label), DI
        0x48, 0xBF,
        byte(j),
        byte(j >> 8),
        byte(j >> 16),
        byte(j >> 24),
        byte(j >> 32),
        byte(j >> 40),
        byte(j >> 48),
        byte(j >> 56),
        // LEAQ 6(PC), SI
        0x48, 0x8d, 0x35, (4 + 2), 0, 0, 0,
        // MOVQ SI, (SP)
        0x48, 0x89, 0x34, 0x24,
        // JMP DI
        0xff, 0xe7,
        // ADDQ $framesize, SP
        0x48, 0x83, 0xc4, (8 + 8),
        // RET
        0xc3,
    }

    executable, err := mmapExecutable(len(code))
    if err != nil {
        panic(err)
    }
    copy(executable, code)
    callJIT(&executable[0])

    munmapExecutable(executable)
}
```

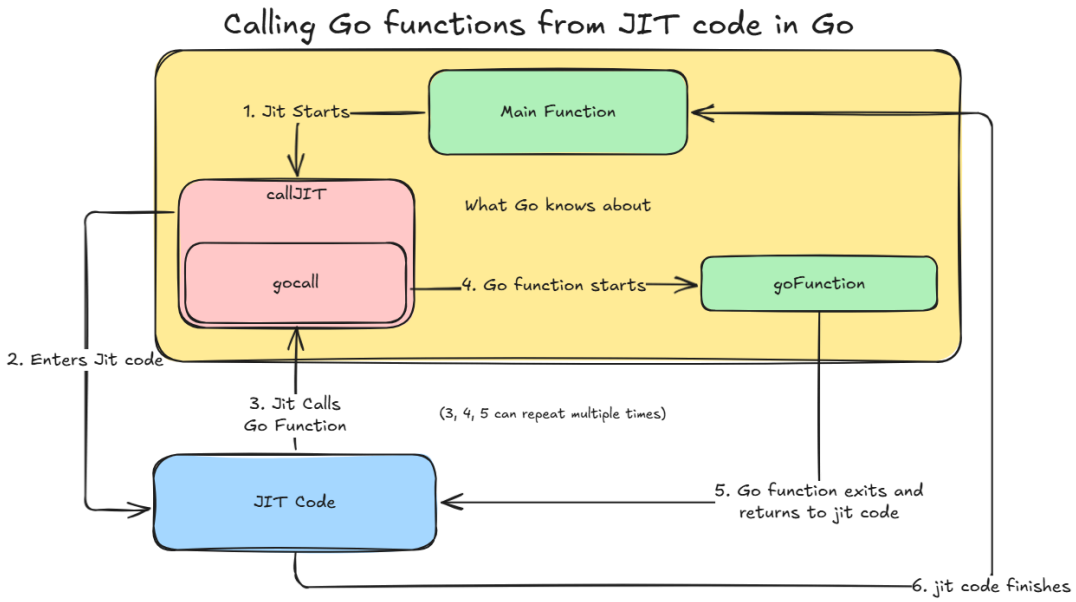
With a pointer to the gocall label, we can now build our CallFunction logic within the JIT code.

1. MOVABS *goFunction, CX: move the pointer to goFunction in RCX
2. MOVABS *gocall, DI: move the pointer to the gocall label into RDI
3. LEAQ 6(PC), SI: the address PC+6 is temporarily placed into SI (PC+6 is the address after the jump to gocall label)
4. MOVQ SI, (SP): the previous address is then placed onto SP

Aaron Balke

return the frame to memory. 8 + 8 is used for the previous BP value stored by Go, and the location where the return address of CALL CX is stored from earlier.

7. Finally, we return.



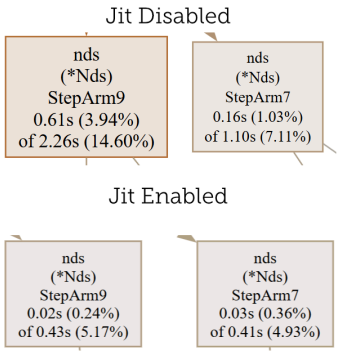
Conclusion

This proof of concept is built upon in gojit. Gojit has been refactored to include a CallFunc(f func) method that supports the assembly code above and the required stack frame handling. The JIT compiler has been used in the guac emulator for GB, GBA, and NDS emulation. It leads to a 2-4x speed increase in CPU emulation. I do recommend checking out the guac emulator if you are interested in a real-life example. It includes the setup of multiple JIT pages, analysis and metrics for JIT compilation thresholds, invalidation, and an LRU cache for invalidating dead JIT compiler blocks.

Cpu Emulation of First 1000 Frames of Mario Kart DS
Without Jit: 3.36 seconds
With Jit: 0.84 seconds
4x speed increase
source: pprof

The Nintendo DS has 2 CPUs, an ARM7 and an ARM9. Both require jit compiling, technically, Guac has 2 jit compilers.

Exact values are not guaranteed. JIT compilation is effectively non-deterministic.



Aaron Balke

Go internal ABI specification

runtime: “unexpected return pc” with JIT-generated code #20123

Proposal: Create an undefined internal calling convention

Calling Go funcs from asm and JITed code